

AI-POWERED NETWORK INTRUSION DETECTION SYSTEM

Software Requirements Specification (SRS)

Document Type	Software Requirements Specification
Project Domain	Artificial Intelligence / Network Security
Primary Application	Network Intrusion Detection (Traffic-Flow Based)
Detection Approach	Machine Learning
Document Version	1.0
Status	Requirements Baseline

Purpose: This document defines the functional, non-functional, technical, security, data, machine-learning, evaluation, and deployment requirements for the AI-Powered Network Intrusion Detection System.

1. Introduction

Network intrusion is a cybersecurity threat in which an attacker attempts to gain unauthorized access to, disrupt, or exfiltrate data from a network by exploiting vulnerabilities in protocols, services, or misconfigured hosts. Intrusions manifest as anomalous traffic patterns such as port scanning, denial-of-service floods, brute-force authentication attempts, botnet command-and-control traffic, and lateral movement between hosts.

The proposed system focuses on automated intrusion detection using network-flow-based machine-learning techniques. Research on datasets such as NSL-KDD, CICIDS2017/2018, and UNSW-NB15 has demonstrated the use of statistical and behavioral flow characteristics with machine-learning models, including ensemble and deep-learning methods, for multi-class attack classification. Recent research has also emphasized real-time detection, explainability, and cross-dataset generalization.

The system is intended to provide a practical network intrusion detection platform while also supporting reproducible experimentation and comparative evaluation of machine-learning models.

2. Purpose

The purpose of the system is to determine whether observed network traffic represents benign activity or a specific category of intrusion using machine-learning-based analysis of flow-level features.

The system shall transform captured or ingested network traffic into a structured representation of flow features, apply a trained classification model, and return a prediction with an associated confidence score and attack-category label.

The system shall also provide an experimental framework for comparing machine-learning algorithms and evaluating their ability to generalize to previously unseen network environments and attack traffic.

3. System Scope

3.1 In Scope

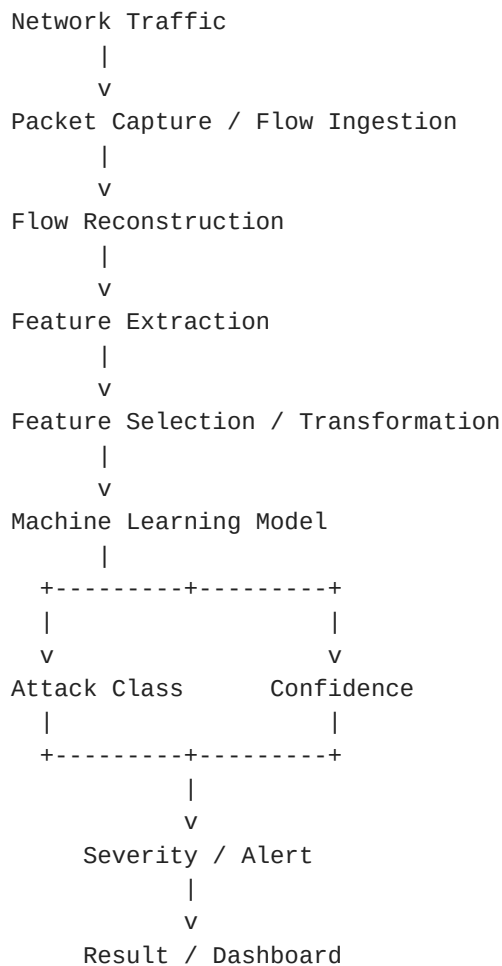
- Network traffic capture and ingestion (live or from packet-capture files)
- Flow reconstruction and session grouping
- Statistical and behavioral flow feature extraction
- Protocol-level feature extraction (TCP/UDP/ICMP, header fields)
- Feature preprocessing and selection
- Machine-learning model training
- Machine-learning model comparison
- Multi-class attack classification (e.g., DoS, probe, R2L, U2R, botnet)
- Intrusion probability / confidence estimation
- Alert severity classification
- Prediction explanation

- Model evaluation
- Cross-dataset validation
- Web-based monitoring dashboard
- REST API
- Alert logging
- Performance monitoring

3.2 Out of Scope

- Automatic blocking or termination of network connections
- Automated exploitation or penetration testing of discovered vulnerabilities
- Deep packet payload decryption of encrypted traffic
- Malware execution or sandboxing
- Unauthorized network scanning
- Guaranteed detection of every zero-day intrusion technique
- Full inline network-gateway deployment in the initial release
- Automatic modification of firewall or router configurations

4. System Overview



The architecture separates traffic ingestion, feature engineering, machine learning, prediction, and presentation so that individual components can be modified or evaluated independently.

5. Functional Requirements

FR-001 — Traffic Ingestion

The system shall provide a mechanism to ingest network traffic either from a live network interface or from a stored packet-capture (PCAP) or flow-export file.

FR-002 — Traffic Validation

The system shall validate ingested traffic sources before analysis and shall reject empty, malformed, unsupported, or corrupted capture inputs.

FR-003 — Flow Reconstruction

The system shall reconstruct traffic into flows or sessions using a consistent flow-key definition (e.g., source/destination IP, port, protocol).

FR-004 — Feature Extraction

The system shall extract measurable characteristics from each flow, including statistical, temporal, and protocol-level characteristics.

FR-005 — Feature Processing

The system shall preprocess extracted features using the same transformation pipeline used during model training.

FR-006 — Feature Selection

The system shall support feature selection or dimensionality reduction when required by the selected experimental methodology.

FR-007 — Model Loading

The production inference service shall load a validated and versioned machine-learning model.

FR-008 — Traffic Classification

The system shall classify a network flow as benign or as belonging to a specific intrusion category. A configurable severity tier may be supported.

FR-009 — Confidence Estimation

The system shall return a probability or confidence score associated with each classification.

FR-010 — Severity Classification

The system shall map the model output to a configurable alert-severity category.

FR-011 — Prediction Explanation

The system shall expose important features or explanatory information associated with the prediction where supported by the selected model.

FR-012 — Alert Record

The system shall optionally record the flow identifier, timestamp, model version, prediction, and relevant evaluation metadata.

FR-013 — Report Generation

The system shall support generation of an analysis report containing the traffic analysis result and associated model information.

FR-014 — Error Handling

The system shall return meaningful error responses for invalid input, missing features, unavailable models, and internal processing failures.

6. Network Flow Feature Requirements

The feature extraction component shall support a configurable flow feature set. The final feature set shall be documented and versioned with the model.

Category	Example Features
Flow statistics	Flow duration, packet count, byte count, packets/sec, bytes/sec
Packet size	Min/max/mean/std packet length, header length
Timing	Inter-arrival time statistics, active/idle time
TCP flags	SYN, ACK, FIN, RST, PSH, URG counts and ratios
Protocol	Protocol type, service/port, connection state
Behavioral	Same-host / same-service connection counts, error rates
Statistical	Entropy and distribution features over payload/header fields

Feature definitions shall be documented so that the training and inference pipelines produce identical feature representations.

7. Machine Learning Requirements

ML-001 — Training Pipeline

The system shall provide a reproducible training pipeline covering data loading, preprocessing, feature preparation, model training, and evaluation.

ML-002 — Baseline Models

The experimental framework shall support baseline models such as Logistic Regression, Decision Tree, Random Forest, SVM, or equivalent baseline classifiers.

ML-003 — Boosting and Deep Models

The framework shall support gradient-boosting models (e.g., XGBoost, LightGBM) and, optionally, deep-learning architectures (e.g., LSTM, CNN) for sequential traffic patterns, for comparative evaluation.

ML-004 — Hyperparameter Optimization

The framework shall support configurable hyperparameter optimization using an appropriate search strategy.

ML-005 — Model Versioning

Every production model shall have an identifiable version associated with its training configuration and feature schema.

ML-006 — Reproducibility

Training experiments shall record dataset versions, feature versions, model parameters, evaluation configuration, and random seeds where applicable.

8. Dataset Requirements

DATA-001 — Dataset Documentation

Every dataset shall have documented source, collection information, class definitions, feature definitions, licensing or usage conditions, and preprocessing procedures.

DATA-002 — Class Labels

The dataset shall distinguish benign traffic and intrusion categories using well-defined labels (e.g., DoS, Probe, R2L, U2R, Botnet, Web Attack).

DATA-003 — Data Quality

The preprocessing pipeline shall identify duplicate flows, missing values, malformed records, inconsistent labels, and other relevant data-quality issues.

DATA-004 — Class Distribution

The class distribution shall be measured and reported. Appropriate balancing or weighting techniques shall be applied where justified given the typical class imbalance in intrusion datasets.

DATA-005 — Data Leakage Prevention

Training, validation, and test data shall be separated in a manner that minimizes duplicate or related flows leaking between partitions, including flows from the same session.

9. Cross-Dataset Validation Requirements

Cross-dataset validation is a core research requirement. The purpose is to determine whether the detection model generalizes beyond the network environment and dataset used for model development.

CV-001 — Independent Dataset Testing

The system shall support evaluation on at least one dataset that is independent of the primary training dataset (e.g., training on CICIDS2017 and testing on CICIDS2018 or UNSW-NB15).

CV-002 — Dataset-Specific Results

Performance results shall be reported separately for each evaluation dataset.

CV-003 — Generalization Analysis

The system shall compare within-dataset and cross-dataset performance and identify significant performance changes.

CV-004 — Feature Stability

The analysis shall investigate whether the selected flow features remain effective across different network environments and datasets.

This requirement is motivated by published research showing that strong performance on a custom or single-source dataset does not necessarily translate to equivalent performance on independent network environments, particularly for less-represented attack classes such as R2L and U2R.

10. Model Evaluation Requirements

Metric	Purpose
Accuracy	Overall classification correctness
Precision	Reliability of positive/intrusion predictions
Recall (Detection Rate)	Ability to identify intrusion samples
F1-score	Balance between precision and recall
Specificity	Ability to correctly identify benign traffic
False Positive Rate	Rate of benign flows incorrectly flagged as intrusions
False Negative Rate	Rate of intrusions missed by the model
ROC-AUC	Ranking/discrimination performance
Per-Class Metrics	Precision/recall per attack category, given class imbalance
Inference Time	Operational prediction latency, relevant for near-real-time detection

EVAL-001 — Confusion Matrix

The system shall generate confusion matrices for applicable classification experiments, including per-attack-class breakdowns.

EVAL-002 — Model Comparison

The experimental framework shall compare models using consistent datasets, features, partitions, and evaluation metrics.

EVAL-003 — Error Analysis

The evaluation shall investigate representative false-positive and false-negative cases, with attention to rare attack classes.

EVAL-004 — Benchmarking

The final model selection shall consider detection performance, generalization, computational requirements, and deployment suitability for near-real-time operation.

11. Explainability Requirements

XAI-001 — Feature Importance

The system shall expose feature-importance information when supported by the selected model.

XAI-002 — Prediction Explanation

The system should provide human-readable information describing which flow characteristics contributed to a given intrusion classification.

XAI-003 — Explanation Consistency

Explanations shall correspond to the model version and feature schema used to produce the prediction.

12. Web Application Requirements

WEB-001 — Monitoring Dashboard

The application shall provide a web dashboard for viewing live and historical traffic classifications.

WEB-002 — Result Display

The interface shall display prediction, attack category, confidence score, severity level, and relevant explanatory information.

WEB-003 — API Integration

The web interface shall communicate with the backend through a defined API.

WEB-004 — Responsive Interface

The interface shall remain usable on common desktop and mobile screen sizes.

WEB-005 — Failure Feedback

The interface shall provide understandable feedback when analysis or ingestion fails.

13. API Requirements

Endpoint	Method	Purpose
/api/health	GET	Service health check
/api/analyze	POST	Analyze submitted flow(s) or capture file
/api/models	GET	Return available model metadata
/api/alerts/{id}	GET	Retrieve a specific alert / analysis report
/api/alerts	GET	List recent alerts with filtering

API-001 — Input Validation

All API inputs shall be validated before processing.

API-002 — Structured Responses

The API shall return consistent machine-readable response structures.

API-003 — Error Codes

The API shall provide appropriate HTTP status codes and meaningful error messages.

14. Non-Functional Requirements

14.1 Performance

NFR-001 — Inference Performance

The production inference pipeline should provide near-real-time response times suitable for monitoring live traffic under normal operating conditions. The actual measured latency and throughput (flows/second) shall be reported as an experimental result.

14.2 Reliability

NFR-002 — Fault Tolerance

Malformed captures, incomplete flows, missing features, model failures, and temporary service failures shall be handled without crashing the application or dropping traffic ingestion silently.

14.3 Maintainability

NFR-003 — Modular Architecture

Frontend, API, flow ingestion, feature extraction, model inference, data access, and evaluation components shall be modular.

14.4 Scalability

NFR-004 — Extensibility

The architecture shall permit additional datasets, feature sets, models, and detection techniques to be added without redesigning the entire system, and shall accommodate increasing traffic volume.

14.5 Portability

NFR-005 — Deployment Portability

The application should support deployment on standard Linux, macOS, or Windows development environments and suitable cloud infrastructure.

15. Security Requirements

SEC-001 — Input Sanitization

User-provided data and ingested traffic metadata shall be validated and safely handled before being processed by application components.

SEC-002 — No Automatic Enforcement

The initial detection pipeline shall not automatically block, drop, or modify live network traffic or device configurations.

SEC-003 — API Protection

Production API endpoints shall implement appropriate authentication, rate limiting, input validation, and access controls where required.

SEC-004 — Sensitive Configuration

Secrets, API keys, database credentials, and signing keys shall not be hard-coded in source code.

SEC-005 — Logging

Security-relevant application errors and operational events shall be logged without unnecessarily storing sensitive payload content.

16. Privacy Requirements

PRIV-001 — Data Minimization

Only traffic metadata and flow features required for detection, evaluation, auditing, or explicitly enabled reporting functionality shall be stored; full packet payload capture shall be limited and justified.

PRIV-002 — Retention

Stored flow records and alert data shall have a configurable retention policy.

PRIV-003 — Access Control

Stored analysis data shall only be accessible to authorized components and users.

17. Hardware Requirements

Component	Minimum	Recommended
CPU	4 cores	6–8 cores
RAM	8 GB	16 GB+
Storage	20 GB	50 GB+
GPU	Not required	Optional (for deep-learning variants)
Network	Monitored network interface / capture access	Dedicated NIC with mirror/SPAN port

GPU acceleration is not a mandatory requirement for the primary flow-feature machine-learning pipeline. CPU-based training and inference are expected to be sufficient for conventional tabular ML models; a GPU is recommended only if deep-learning sequence models are used.

18. Software Requirements

Category	Recommended Technology
Language	Python 3.x
Data Processing	Pandas, NumPy
Machine Learning	Scikit-learn
Boosting / Deep Learning	XGBoost, LightGBM, optional PyTorch/TensorFlow
Traffic Capture	Scapy, Zeek, or CICFlowMeter for flow export
Visualization	Matplotlib
Backend	FastAPI
Frontend	HTML, CSS, JavaScript
Database	MongoDB or PostgreSQL
Model Serialization	Joblib
Version Control	Git

19. System Constraints

- Detection quality depends on the quality and representativeness of training traffic data.
- The model cannot guarantee detection of previously unseen or zero-day intrusion techniques.
- Dataset imbalance across attack classes may influence evaluation results.
- Cross-dataset performance may be lower than within-dataset performance.
- Encrypted traffic limits the availability of payload-based features; flow-level features remain usable.

- High traffic volume may impose throughput and latency constraints on real-time inference.
- Model predictions should be treated as automated risk assessments rather than absolute security guarantees.

20. Future Extensions

- Deep packet inspection for supported unencrypted protocols
- Sequence-based deep learning (LSTM/Transformer) for temporal attack patterns
- Graph-based analysis of host-to-host communication patterns
- Automated response integration (e.g., firewall rule suggestions)
- Real-time threat-intelligence feed integration
- Adversarial robustness testing against evasion techniques
- Continuous model retraining
- Online / incremental learning
- Distributed sensor deployment across multiple network segments

21. Acceptance Criteria

- Network traffic can be ingested through the application, live or from a capture file.
- Invalid or malformed input is rejected safely.
- Traffic is reconstructed into flows and converted into the required feature representation.
- A trained model produces a classification.
- An intrusion probability or confidence score is returned.
- A configurable severity level is displayed.
- The model version is identifiable.
- Model performance is measured using multiple appropriate metrics, including per-class metrics.
- At least one independent dataset is available for cross-dataset validation.
- False-positive and false-negative cases are analyzed.
- The API returns structured responses.
- The web dashboard displays analysis results correctly.
- The system handles application errors without terminating unexpectedly.
- The final model and feature pipeline are reproducible from documented configuration.

22. Requirements Traceability Matrix

Requirement Area	Implementation Component	Validation
Traffic Input	Ingestion module + API	Valid/invalid capture tests
Flow Reconstruction	Flow module	Flow-key unit tests
Feature Extraction	Feature pipeline	Feature unit tests
Model Training	Training pipeline	Training experiment
Model Comparison	Evaluation pipeline	Benchmark results
Prediction	Inference service	Prediction tests
Explainability	XAI module	Feature importance
Cross-Dataset	Evaluation pipeline	Independent test dataset
Security	API/security layer	Security test cases
Performance	Inference service	Latency/throughput benchmark
Reporting	Report module	Generated report validation

23. Research References

The reference list below is a placeholder structure matching common IEEE-Access-style network intrusion detection literature (NSL-KDD, CICIDS2017/2018, UNSW-NB15 based studies). Replace with the specific papers cited in your project before submission.

- Author(s), "Title of NSL-KDD / CICIDS-based intrusion detection study," IEEE Access, Vol. XX, Year, pp. XXXXX–XXXXX. DOI: 10.1109/ACCESS.XXXX.XXXXXXX.
- Author(s), "Title of cross-dataset generalization study for network intrusion detection," IEEE Access, Vol. XX, Year, pp. XXXXX–XXXXX. DOI: 10.1109/ACCESS.XXXX.XXXXXXX.
- Author(s), "Title of deep-learning-based intrusion detection study," IEEE Access, Vol. XX, Year, pp. XXXXX–XXXXX. DOI: 10.1109/ACCESS.XXXX.XXXXXXX.
- Author(s), "Title of explainable AI for network security study," IEEE Access, Vol. XX, Year, pp. XXXXX–XXXXX. DOI: 10.1109/ACCESS.XXXX.XXXXXXX.